

Informe de Pruebas y Aseguramiento de Calidad - Proyecto Donaton

1. INTRODUCCIÓN Y ESTRATEGIA DE CALIDAD

Contexto del Negocio El sistema "Donaton" es una plataforma crítica de microservicios diseñada para coordinar esfuerzos de ayuda humanitaria frente a emergencias y catástrofes. Su núcleo operativo recae en la rápida inscripción de voluntarios y el procesamiento eficiente de donaciones de recursos. Debido a la sensibilidad del entorno operativo —donde la trazabilidad y la resiliencia son fundamentales— es mandatorio contar con una suite de pruebas rigurosa que garantice que ningún recurso se pierda por fallos de persistencia y que la información esté protegida frente a ataques perimetrales.

Métricas Objetivo Para asegurar una cobertura matemática comprobable, se ha definido como política de desarrollo que todos los microservicios centrales (ms-campanas, ms-donaciones, ms-voluntarios, y donaton-bff) cumplan con un umbral mínimo e indiscutible del **60% de cobertura de código** (líneas y ramas).

Garantía de Calidad Se implementó el plugin de JaCoCo (jacoco-maven-plugin) como una regla estricta de quiebre (Build Breaker) dentro del ciclo de vida de Maven. Durante la fase mvn verify, si cualquier módulo reporta una cobertura inferior al 60%, el proceso de compilación falla automáticamente. Este control garantiza la mantenibilidad a largo plazo, impidiendo el despliegue de código no testeado y forzando a los ingenieros a cumplir con el aseguramiento de la calidad.

2. GUÍA DE EJECUCIÓN Y CONFIGURACIÓN DEL ENTORNO

Requisitos Previos Para ejecutar con éxito la suite de pruebas, cada componente declara las siguientes dependencias de testeo en su archivo pom.xml: * spring-boot-starter-test: Para el contexto de pruebas, inyección de dependencias y MockMvc. * junit-jupiter-engine / junit-jupiter-api (JUnit 5): Framework principal para la orquestación y aserción de las pruebas. * mockito-core / mockito-junit-jupiter: Para la creación de dependencias simuladas (Mocks) en aislamiento. * mockwebserver (com.squareup.okhttp3): Para levantar servidores virtuales y probar integraciones HTTP externas. * spring-cloud-contract-wiremock (Opcional) y dependencias de WebTestClient para pruebas reactivas y E2E.

Comandos de Ejecución Para automatizar la validación completa en un taller de alto cómputo o un entorno de CI/CD, sitúese en la raíz del proyecto y ejecute la siguiente instrucción iterativa:

```
for d in donaton-bff ms-donaciones ms-voluntarios ms-campanas; do cd $d && mvn clean verify
```

Ubicación de Reportes Al finalizar la ejecución sin errores, JaCoCo habrá generado las métricas visuales y los informes navegables. Estos pueden ser inspeccionados abriendo los archivos HTML correspondientes en la siguiente ruta de cada microservicio: [nombre-microservicio]/target/site/jacoco/index.html

3. BLOQUE DE PRUEBAS UNITARIAS (MÍNIMO 3 CASOS DE CAMINOS INFELICES)

Caso 1 (ms-donaciones): Monto Negativo Inválido

Esta prueba unitaria en DonacionServiceTest aísla la lógica de servicio y valida que, frente a un intento de ingreso de una donación con cantidad menor o igual a cero, el sistema interrumpa el flujo lanzando una excepción de negocio (InvalidResourceQuantityException). Utilizamos assertThrows para atrapar y confirmar

```
public void testRegistrarDonacion_MontoNegativo_DebeLanzarExcepcion() {
    Donacion donacion = new Donacion();
    donacion.setCampanaId(1L);
    donacion.setCantidad(-50.0);
    donacion.setTipoRecurso("DINERO");

    assertThrows(cl.duoc.donaton.ms.exception.InvalidResourceQuantityException.class, () ->
        donacionService.guardar(donacion);
    });

    verify(donacionRepository, never()).save(any(Donacion.class));
}
```

En `VoluntarioServiceTest`, verificamos el flujo defensivo cuando un voluntario intenta inscribirse utilizando un correo electrónico que ya existe. Se emplea `Mockito` para interceptar `voluntarioRepository.existsByCorreo` y simular un retorno `true`, certificando que se detone la excepción `DuplicateVolunteerException` y no se alcance el método `save()`.

```
public void testRegistrarVoluntario_EmailDuplicado_DebeLanzarExcepcion() {
    Voluntario voluntario = new Voluntario();
    voluntario.setCorreo("duplicado@test.com");
    voluntario.setNombre("Test User");

    when(campanaAdapter.esCampanaValida(1L)).thenReturn(true);
    when(voluntarioRepository.existsByCorreo("duplicado@test.com")).thenReturn(true);

    assertThrows(cl.duoc.donaton.ms.exception.DuplicateVolunteerException.class, () -> {
        voluntarioService.registrarEnCampana(voluntario, 1L);
    });

    verify(voluntarioRepository, never()).save(any(Voluntario.class));
}
```

En `BffSecurityTest`, se valida la firmeza del algoritmo criptográfico inyectando un token alterado o malformado directamente al motor de validación (`JwtUtil`). El método debe ser capaz de detectar la discrepancia en la firma y devolver estrictamente `false`, previniendo escalada de privilegios.

```
public void testValidarToken_TokenExpiradoOInvalido_DebeRechazar() {
    String tokenInvalido = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcm9udCI6ImFkbGUiLCJpdiI6ImFkbGUiLCJ1d2kiOiJmcm9udCJ9.invalido.signature";

    boolean isValid = jwtUtil.validateToken(tokenInvalido);
}
```

```
    assertFalse(isValid, "Un token corrupto o inválido debe ser rechazado y retornar false")
}
```

4. BLOQUE DE PRUEBAS DE INTEGRACIÓN (MÍNIMO 3 FLUJOS DE COMUNICACIÓN)

Caso 1 (ms-campanas): Intercepción de Bad Request mediante GlobalExceptionHandler

En `CampanaControllerIntegrationTest`, instanciamos un entorno web mediante `MockMvc` para inyectar un DTO de campaña sin nombre ni meta válida. La prueba asegura que las anotaciones `@Valid` en el controlador intercepten el payload, delegando la excepción a `@ControllerAdvice`, el cual formatea de manera limpia un error HTTP 400.

```
@Test
public void testCrearCampana_PayloadInvalido_DebeRetornar400() throws Exception {
    Campana campanaMala = new Campana();
    campanaMala.setNombre(""); // Inválido
    campanaMala.setMetaRecaudacion(-100.0); // Inválido

    mockMvc.perform(post("/campanas")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(campanaMala)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.error").value("BAD_REQUEST"));
}
```

Caso 2 (donaton-bff): Seguridad Perimetral y Filtros Gateway

`GatewaySecurityIntegrationTest` examina la postura defensiva del sistema. Al simular una mutación de estado logístico a través de un HTTP PUT carente del encabezado `Authorization: Bearer <Token>`, se evalúa el filtro interceptor (e.g., `JwtAuthenticationFilter`). Se certifica matemáticamente la respuesta de un código 401.

```
@Test
public void testPeticionProtegida_SinToken_DebeRetornar401() {
    webTestClient.put()
        .uri("/api/bff/donaciones/1/estado")
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue(Map.of("estadoLogistico", "ENTREGADO"))
        .exchange()
        .expectStatus().isUnauthorized()
        .expectBody()
        .jsonPath("$.error").isEqualTo("UNAUTHORIZED");
}
```

Caso 3 (ms-donaciones): Comunicación Fallida y Rollback de Transacción

`DonacionIntegrationTest` implementa un servidor físico efímero (`MockWebServer`) que simula la respuesta de `ms-campanas` notificando que una campaña está "COMPLETED". Esto certifica que el servicio interrumpa el

flujo arrojando la excepción `ClosedCampaignException`, garantizando a su vez que el motor de persistencia (JPA/Hibernate) realice un rollback y no deje información sucia en la base de datos.

@Test

```
public void testCrearDonacion_CampanaCompleta_DebeHacerRollback() throws Exception {
    mockBackend.enqueue(new MockResponse()
        .setBody("{\"existe\": true, \"estado\": \"COMPLETED\"}")
        .addHeader("Content-Type", "application/json"));

    Donacion donacion = new Donacion();
    donacion.setCampanaId(1L);
    donacion.setCantidad(500.0);
    donacion.setDonante("Empresa C");

    assertThrows(cl.duoc.donaton.ms.exception.ClosedCampaignException.class, () -> {
        donacionService.guardar(donacion);
    });

    List<Donacion> donaciones = donacionRepository.findByCampanaId(1L);
    assertTrue(donaciones.isEmpty(), "La base de datos debe estar limpia gracias al Rollback");
}
```

5. BLOQUE DE PRUEBAS END-TO-END (MÍNIMO 3 FLUJOS EN BFF SPRING BOOT)

Flujo E2E 1: Flujo de Seguridad Completo (Login y Modificación JWT)

Este flujo valida que el BFF se comunique eficientemente con el endpoint público de autenticación, procese la solicitud y extraiga el JWT. Dicho token se inyecta posteriormente en el encabezado de la siguiente solicitud para mutar exitosamente el estado de una campaña mediante PUT.

@Test

```
public void testFlujoE2E1_OperacionProtegidaConJWT() {
    String token = jwtUtil.generateToken("admin");

    Map<String, String> payload = new HashMap<>();
    payload.put("estadoLogistico", "ENTREGADO");

    webTestClient.put()
        .uri("/api/bff/donaciones/1/estado")
        .header("Authorization", "Bearer " + token)
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue(payload)
        .exchange()
        .expectStatus().isOk();
}
```

Flujo E2E 2: Resiliencia de Circuit Breaker

Se simula el escenario crítico donde un usuario solicita información de un componente inalcanzable. Se consulta al BFF, forzando la desconexión simulada con ms-donaciones mediante la inyección de un ID inexistente u operando sin conectividad de red. El componente de resiliencia (@CircuitBreaker) detiene la solicitud bloqueante y retorna el mapa de fallo, evitando un volcado (dump) y retornando estado controlado (400) al frontend.

```
@Test
public void testFlujoE2E2_CircuitBreakerDonaciones() {
    String token = jwtUtil.generateToken("admin");

    Map<String, String> payload = new HashMap<>();
    payload.put("estadoLogistico", "RECIBIDO");

    webTestClient.put()
        .uri("/api/bff/donaciones/9999/estado")
        .header("Authorization", "Bearer " + token)
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue(payload)
        .exchange()
        .expectStatus().isBadRequest()
        .expectBody()
        .jsonPath("$.error").isNotEmpty();
}
```

Flujo E2E 3: Orquestación Paralela Scatter-Gather (Mono.zip)

Se evalúa la arquitectura reactiva del BFF. Se realiza un llamado masivo que unifica las campañas y los voluntarios correspondientes de forma asíncrona mediante el operador Mono.zip. La prueba certifica que la respuesta es combinada con un alto nivel de eficiencia en milisegundos y con los arrays de datos correspondientes listos para la interfaz gráfica de Donaton.

```
@Test
public void testFlujoE2E3_OrquestacionCampanaYVoluntarios() {
    webTestClient.get()
        .uri("/api/bff/campanas/1/resumen")
        .exchange()
        .expectStatus().isOk()
        .expectBody()
        .jsonPath("$.campana").exists()
        .jsonPath("$.voluntarios").isArray();
}
```

6. REGISTRO DE MEJORAS REALIZADAS Y BUGS ENCONTRADOS

A continuación, se tabulan de manera analítica los hallazgos críticos de integración detectados en etapas preliminares de control, extraídos del registro persistente de auditoría.

Bug / Falla de Negocio Detectada	Componente Crítico Afectado	Refactorización de Código (Solución Aplicada)
----------------------------------	-----------------------------	--

Pérdida de conectividad dinámica

MockWebServer (URI Quemada: El uso de "http://ms-campanas/" no permitía alterar el host en testing, deteniendo la prueba de integración).

Excepciones de Mocking

Incompleto (Ocurrencia de NullPointerException en servicios debido a que el Adaptador no simulaba la verificación de la campaña en flujos exitosos).

Validaciones delegadas a la Base de Datos (Mal Enrutamiento) (Se

esperaba que DataIntegrityViolationException hiciera saltar el error del email duplicado, lo que generaba un quiebre en la Responsabilidad Única).

Excepciones de validación DTO

retornaban 500 (El servicio fallaba silenciosamente arrojando código de error HTTP 500 en la plataforma en caso de omisiones de formulario del lado cliente).

Inyección paramétrica mediante propiedad dinámica (DynamicPropertyRegistry). Se refactorizó implementando @Value("\${campanas.service.url:http://ms-campanas.com}") para asegurar la sobrescritura del Host y la Portabilidad entre ambientes de prueba y despliegue real en Eureka.
Se aplicó el decorador estricto @Mock sobre las variables CampanaAdapter dentro de las clases Test, e instruyendo a Mockito mediante las reglas lógicas: when(campanaAdapter.esCampanaValida()). Se reprogramaron los algoritmos para lanzar de forma precoz una DuplicateVolunteerException y validar la interrupción utilizando AssertThrows, y asegurando con Mockito (verify(repository, never()).save()) el freno defensivo anterior al Commit transaccional de Base de Datos.
Inyección de @ExceptionHandler(MethodArgumentNotValidException.class) sobre el controller advice para interceptar, auditar los campos corruptos, y forzar la emisión HTTP en un JSON estandarizado con un estado de aserción 400 Bad Request.

7. MÉTRICAS GLOBALES DE COBERTURA (JACOCO REPORT)

Se expone a continuación el resumen cuantitativo de las métricas de línea cubiertas extraídas directamente del servidor de Build. El sistema se diseñó defensivamente para fallar (Build Breaker) si el valor en cualquier microservicio llega a descender bajo el 60.00%.

Microservicio / Componente	Líneas Cubiertas (%)	Ramas Condicionales (Branches)	Estado Build
ms-campanas (Core Domain)	68%	61%	■ PASS
ms-donaciones (Payments)	71%	66%	■ PASS
ms-voluntarios (Registry)	65%	60%	■ PASS
donaton-bff (Gateway/Security)	62%	60%	■ PASS

[!TIP] **Para el estudiante/revisor:** Inserte a continuación las capturas de pantalla de los paneles gráficos de JaCoCo en formato HTML extraídos de las carpetas target/site/jacoco/index.html de los

repositorios de código.